Module 7 : Model Validation with Walk-Forw Testing

Learning Objectives

By the end of this module, you will:

1. Understand why traditional cross-validation fails for time series
2. Master the `TimeSeriesSplit` and `WalkForwardValidator` classes
3. Implement expanding and rolling window validation
4. Use gap periods to prevent look-ahead bias
5. Interpret validation fold results and aggregate metrics
6. Compare multiple models fairly with walk-forward testing
7. Detect overfitting and data leakage

The Critical Problem

"Validation is where most quantitative strategies die."

You can have:

- Perfect feature engineering ✓
- Sophisticated models ✓
- Clean data ✓

But if your validation methodology is flawed, your backtested performance will be **dangerously misleading**.

In [1]:
```
# Standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_s
import warnings
warnings.filterwarnings('ignore')

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.precision', 4)

print("✓ Environment configured")
```

```
✓ Environment configured
```

In [2]:
```python
# Import fair_value_toolkit
import sys
sys.path.insert(0, '../../src')

from fair_value_toolkit.validation import (
    TimeSeriesSplit,
    WalkForwardValidator,
    ValidationFold,
    ValidationResult,
    detect_data_leakage,
    calculate_model_decay
)
from fair_value_toolkit.models import (
    InventoryFairValueModel,
    SupplyDemandModel,
    EnsembleFairValueModel
)
from fair_value_toolkit.features import create_commodity_features

print("✓ fair_value_toolkit imported")
```

```
✓ fair_value_toolkit imported
```

Part 1 : Why Cross-Validation Fails for Time Series

The Standard Cross-Validation Trap

Sklearn's `KFold` and `StratifiedKFold` randomly split data into folds. This is **catastrophic wrong** for time series because:

1. **Training on future, testing on past** - Impossible in production
2. **Temporal dependencies** - Serial correlation violates independence
3. **Look-ahead bias** - Future information leaks into training

Let's demonstrate this problem.

In [3]:
```python
# Create synthetic time series with trend and autocorrelation
np.random.seed(42)
n = 1000
dates = pd.date_range('2020-01-01', periods=n, freq='D')

# Price with strong trend and autocorrelation
trend = np.linspace(50, 80, n)
ar_component = np.zeros(n)
for i in range(1, n):
    ar_component[i] = 0.7 * ar_component[i-1] + np.random.normal(0, 2)

price = trend + ar_component

# Create simple feature: lagged price
df = pd.DataFrame({
    'price': price,
    'price_lag1': pd.Series(price).shift(1),
    'price_lag5': pd.Series(price).shift(5),
    'price_lag21': pd.Series(price).shift(21),
}, index=dates).dropna()
```

```python
print(f"Dataset shape: {df.shape}")
print(f"Date range: {df.index[0].date()} to {df.index[-1].date()}")

# Plot
plt.figure(figsize=(14, 5))
plt.plot(df.index, df['price'], alpha=0.7)
plt.xlabel('Date')
plt.ylabel('Price')
plt.title('Synthetic Time Series with Trend and Autocorrelation')
plt.grid(True, alpha=0.3)
plt.show()

# Check autocorrelation
from statsmodels.graphics.tsaplots import plot_acf
fig, ax = plt.subplots(figsize=(12, 4))
plot_acf(df['price'], lags=50, ax=ax)
ax.set_title('Autocorrelation Function (Strong temporal dependence)')
plt.show()
```

```
Dataset shape: (0, 4)
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[3], line 23
     15 df = pd.DataFrame({
     16     'price': price,
     17     'price_lag1': pd.Series(price).shift(1),
     18     'price_lag5': pd.Series(price).shift(5),
     19     'price_lag21': pd.Series(price).shift(21),
     20 }, index=dates).dropna()
     22 print(f"Dataset shape: {df.shape}")
---> 23 print(f"Date range: {df.index[0].date()} to {df.index[-1].date()}")
     25 # Plot
     26 plt.figure(figsize=(14, 5))

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/indexes/base.py:5401, in Index.__getitem
f, key)
   5398 if is_integer(key) or is_float(key):
   5399     # GH#44051 exclude bool, which would return a 2d ndarray
   5400     key = com.cast_scalar_indexer(key)
-> 5401     return getitem(key)
   5403 if isinstance(key, slice):
   5404     # This case is separated from the conditional above to avoid
   5405     # pessimization com.is_bool_indexer and ndim checks.
   5406     return self._getitem_slice(key)

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/datetimelike.py:398, in DatetimeL
ayMixin.__getitem__(self, key)
    391 """
    392 This getitem defers to the underlying array, which by-definition can
    393 only handle list-likes, slices, and integer scalars
    394 """
    395 # Use cast as we know we will get back a DatetimeLikeArray or DTScala
    396 # but skip evaluating the Union at runtime for performance
    397 # (see https://github.com/pandas-dev/pandas/pull/44624)
--> 398 result = cast("Union[Self, DTScalarOrNaT]", super().__getitem__(key))
    399 if lib.is_scalar(result):
    400     return result

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/_mixins.py:284, in NDArrayBackedE
onArray.__getitem__(self, key)
    278 def __getitem__(
    279     self,
    280     key: PositionalIndexer2D,
    281 ) -> Self | Any:
    282     if lib.is_integer(key):
    283         # fast-path
--> 284         result = self._ndarray[key]
    285         if self.ndim == 1:
    286             return self._box_func(result)

IndexError: index 0 is out of bounds for axis 0 with size 0
```

In [4]:

```python
# Compare wrong (random CV) vs correct (walk-forward)
from sklearn.model_selection import KFold

X = df[['price_lag1', 'price_lag5', 'price_lag21']].values
y = df['price'].values
```

```python
# WRONG: Random K-Fold
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
random_scores = []

for train_idx, test_idx in kfold.split(X):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    model = LinearRegression()
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    random_scores.append(score)

# CORRECT: Time Series Split
from sklearn.model_selection import TimeSeriesSplit as SklearnTSSplit
tscv = SklearnTSSplit(n_splits=5)
temporal_scores = []

for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    model = LinearRegression()
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    temporal_scores.append(score)

print("\n" + "="*60)
print("PERFORMANCE COMPARISON")
print("="*60)
print(f"\nWRONG (Random K-Fold):")
print(f"  Mean R²: {np.mean(random_scores):.4f} ± {np.std(random_scores):
print(f"  Fold scores: {[f'{s:.4f}' for s in random_scores]}")

print(f"\nCORRECT (Time Series Split):")
print(f"  Mean R²: {np.mean(temporal_scores):.4f} ± {np.std(temporal_scor
print(f"  Fold scores: {[f'{s:.4f}' for s in temporal_scores]}")

print(f"\n⚠️  PERFORMANCE GAP: {(np.mean(random_scores) - np.mean(tempora
print("="*60)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[4], line 11
      8 kfold = KFold(n_splits=5, shuffle=True, random_state=42)
      9 random_scores = []
---> 11 for train_idx, test_idx in kfold.split(X):
     12     X_train, X_test = X[train_idx], X[test_idx]
     13     y_train, y_test = y[train_idx], y[test_idx]

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/sklearn/model_selection/_split.py:404, in
_BaseKFold.split(self, X, y, groups)
    402 n_samples = _num_samples(X)
    403 if self.n_splits > n_samples:
--> 404     raise ValueError(
    405         (
    406             "Cannot have number of splits n_splits={0} greater"
    407             " than the number of samples: n_samples={1}."
    408         ).format(self.n_splits, n_samples)
    409     )
    411 for train, test in super().split(X, y, groups):
    412     yield train, test

ValueError: Cannot have number of splits n_splits=5 greater than the number o
samples: n_samples=0.
```

Why the Huge Difference?

Random cross-validation gives you **impossibly good results** because:

1. **Information leakage**: Training on `2023` data, testing on `2020` data
2. **Similar samples**: Adjacent time points are highly correlated
3. **Unrealistic scenario**: You'll never have future data in production

The lesson: Always use walk-forward validation for time series!

Part 2: Understanding Walk-Forward Validation

Walk-forward validation mimics production:

```
Timeline: |--train1--|gap|--test1--||--train2--|gap|--test2--||--
train3--|gap|--test3--|
          ↑                          ↑                          ↑
          Fit model                Predict                    Refit
```

Key Concepts

1. **Training Window**:
   - Expanding: Grows from start (uses all history)
   - Rolling: Fixed size (uses recent N periods)
2. **Gap Period**:
   - Buffer between train and test
   - Prevents leakage from data revisions

- Typical: 1 - 5 days for daily data

3. **Test Window**:

- Out-of-sample period
- Should match your trading horizon

In [5]:
```
# Visualize walk-forward splits
def visualize_splits(splitter, df, title):
    """Visualize train/test splits."""
    folds = splitter.get_folds(df)

    fig, ax = plt.subplots(figsize=(14, 6))

    for i, fold in enumerate(folds):
        # Train period
        ax.barh(i, (fold.train_end - fold.train_start).days,
               left=fold.train_start, height=0.4,
               color='steelblue', alpha=0.7, label='Train' if i == 0 else

        # Gap period
        gap_width = (fold.test_start - fold.train_end).days
        if gap_width > 0:
            ax.barh(i, gap_width,
                   left=fold.train_end, height=0.4,
                   color='yellow', alpha=0.5, label='Gap' if i == 0 else

        # Test period
        ax.barh(i, (fold.test_end - fold.test_start).days,
               left=fold.test_start, height=0.4,
               color='coral', alpha=0.7, label='Test' if i == 0 else '')

    ax.set_xlabel('Date')
    ax.set_ylabel('Fold')
    ax.set_title(title)
    ax.set_yticks(range(len(folds)))
    ax.set_yticklabels([f'Fold {i}' for i in range(len(folds))])
    ax.legend(loc='upper left')
    ax.grid(True, alpha=0.3, axis='x')
    plt.tight_layout()
    return fig

# Create expanding window splitter
expanding_splitter = TimeSeriesSplit(
    n_splits=5,
    train_period='180D',  # Initial training: 180 days
    test_period='60D',    # Test period: 60 days
    gap_period='5D',      # Gap: 5 days
    expanding=True
)

fig = visualize_splits(expanding_splitter, df, 'Expanding Window Walk-For
plt.show()

print("\nExpanding Window Folds:")
for fold in expanding_splitter.get_folds(df):
    print(fold)
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[5], line 45
     36 # Create expanding window splitter
     37 expanding_splitter = TimeSeriesSplit(
     38     n_splits=5,
     39     train_period='180D',  # Initial training: 180 days
   (...)     42     expanding=True
     43 )
---> 45 fig = visualize_splits(expanding_splitter, df,
                                                       )
     46 plt.show()
     48 print("\nExpanding Window Folds:")

Cell In[5], line 4, in visualize_splits(splitter, df, title)
      2 def visualize_splits(splitter, df, title):
      3     """Visualize train/test splits."""
----> 4     folds = splitter.get_folds(df)
      6     fig, ax = plt.subplots(figsize=(14, 6))
      8     for i, fold in enumerate(folds):
      9         # Train period

File /home/user/claude-template/fair-value-commodities-course/notebooks/
07_walk_forward_validation/../../src/fair_value_toolkit/validation.py:210, in
eriesSplit.get_folds(self, X)
    207 dates = X.index.sort_values()
    208 folds = []
--> 210 for fold_num, (train_idx, test_idx) in enumerate(self.split(X)):
    211     fold = ValidationFold(
    212         fold_number=fold_num,
    213         train_start=dates[train_idx[0]],
   (...)    219         gap_days=self.gap_period.days,
    220     )
    221     folds.append(fold)

File /home/user/claude-template/fair-value-commodities-course/notebooks/
07_walk_forward_validation/../../src/fair_value_toolkit/validation.py:147, in
eriesSplit.split(self, X)
    145 # Calculate fold boundaries
    146 total_test_period = self.test_period * self.n_splits
--> 147 data_range = dates[-1] - dates[0]
    149 if self.expanding:
    150     # Expanding window: train starts from beginning, test moves forwa
    151     min_train_end = dates[0] + self.train_period

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/indexes/base.py:5401, in Index.__getitem
f, key)
   5398 if is_integer(key) or is_float(key):
   5399     # GH#44051 exclude bool, which would return a 2d ndarray
   5400     key = com.cast_scalar_indexer(key)
-> 5401     return getitem(key)
   5403 if isinstance(key, slice):
   5404     # This case is separated from the conditional above to avoid
   5405     # pessimization com.is_bool_indexer and ndim checks.
   5406     return self._getitem_slice(key)

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/datetimelike.py:398, in DatetimeL
ayMixin.__getitem__(self, key)
```

```
    391 """
    392 This getitem defers to the underlying array, which by-definition can
    393 only handle list-likes, slices, and integer scalars
    394 """
    395 # Use cast as we know we will get back a DatetimeLikeArray or DTScala
    396 # but skip evaluating the Union at runtime for performance
    397 # (see https://github.com/pandas-dev/pandas/pull/44624)
--> 398 result = cast("Union[Self, DTScalarOrNaT]", super().__getitem__(key))
    399 if lib.is_scalar(result):
    400     return result

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/_mixins.py:284, in NDArrayBackedE
onArray.__getitem__(self, key)
    278 def __getitem__(
    279     self,
    280     key: PositionalIndexer2D,
    281 ) -> Self | Any:
    282     if lib.is_integer(key):
    283         # fast-path
--> 284         result = self._ndarray[key]
    285         if self.ndim == 1:
    286             return self._box_func(result)

IndexError: index -1 is out of bounds for axis 0 with size 0
```

In [6]:

```python
# Create rolling window splitter
rolling_splitter = TimeSeriesSplit(
    n_splits=5,
    train_period='180D',  # Rolling window: 180 days
    test_period='60D',
    gap_period='5D',
    expanding=False  # Rolling window
)

fig = visualize_splits(rolling_splitter, df, 'Rolling Window Walk-Forward
plt.show()

print("\nRolling Window Folds:")
for fold in rolling_splitter.get_folds(df):
    print(fold)
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[6], line 10
      1 # Create rolling window splitter
      2 rolling_splitter = TimeSeriesSplit(
      3     n_splits=5,
      4     train_period='180D',  # Rolling window: 180 days
   (...)      7     expanding=False  # Rolling window
      8 )
---> 10 fig = visualize_splits(rolling_splitter, df,
            )
     11 plt.show()
     13 print("\nRolling Window Folds:")

Cell In[5], line 4, in visualize_splits(splitter, df, title)
      2 def visualize_splits(splitter, df, title):
      3     """Visualize train/test splits."""
----> 4     folds = splitter.get_folds(df)
      6     fig, ax = plt.subplots(figsize=(14, 6))
      8     for i, fold in enumerate(folds):
      9         # Train period

File /home/user/claude-template/fair-value-commodities-course/notebooks/
07_walk_forward_validation/../../src/fair_value_toolkit/validation.py:210, in
eriesSplit.get_folds(self, X)
    207 dates = X.index.sort_values()
    208 folds = []
--> 210 for fold_num, (train_idx, test_idx) in enumerate(self.split(X)):
    211     fold = ValidationFold(
    212         fold_number=fold_num,
    213         train_start=dates[train_idx[0]],
   (...)    219         gap_days=self.gap_period.days,
    220     )
    221     folds.append(fold)

File /home/user/claude-template/fair-value-commodities-course/notebooks/
07_walk_forward_validation/../../src/fair_value_toolkit/validation.py:147, in
eriesSplit.split(self, X)
    145 # Calculate fold boundaries
    146 total_test_period = self.test_period * self.n_splits
--> 147 data_range = dates[-1] - dates[0]
    149 if self.expanding:
    150     # Expanding window: train starts from beginning, test moves forwa
    151     min_train_end = dates[0] + self.train_period

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/indexes/base.py:5401, in Index.__getitem
f, key)
   5398 if is_integer(key) or is_float(key):
   5399     # GH#44051 exclude bool, which would return a 2d ndarray
   5400     key = com.cast_scalar_indexer(key)
-> 5401     return getitem(key)
   5403 if isinstance(key, slice):
   5404     # This case is separated from the conditional above to avoid
   5405     # pessimization com.is_bool_indexer and ndim checks.
   5406     return self._getitem_slice(key)

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/datetimelike.py:398, in DatetimeL
ayMixin.__getitem__(self, key)
```

```
    391 """
    392 This getitem defers to the underlying array, which by-definition can
    393 only handle list-likes, slices, and integer scalars
    394 """
    395 # Use cast as we know we will get back a DatetimeLikeArray or DTScala
    396 # but skip evaluating the Union at runtime for performance
    397 # (see https://github.com/pandas-dev/pandas/pull/44624)
--> 398 result = cast("Union[Self, DTScalarOrNaT]", super().__getitem__(key))
    399 if lib.is_scalar(result):
    400     return result

File /home/user/claude-template/fair-value-commodities-course/fv-env/lib/
python3.11/site-packages/pandas/core/arrays/_mixins.py:284, in NDArrayBackedE
onArray.__getitem__(self, key)
    278 def __getitem__(
    279     self,
    280     key: PositionalIndexer2D,
    281 ) -> Self | Any:
    282     if lib.is_integer(key):
    283         # fast-path
--> 284         result = self._ndarray[key]
    285         if self.ndim == 1:
    286             return self._box_func(result)

IndexError: index -1 is out of bounds for axis 0 with size 0
```

Expanding vs Rolling: When to Use Each?

Expanding Window:

- ✓ More data = potentially better estimates
- ✓ Good for stable, long-term relationships
- ✗ Includes ancient history that may be irrelevant
- ✗ Slower to adapt to regime changes

Rolling Window:

- ✓ Adapts quickly to new market regimes
- ✓ Focuses on recent, relevant data
- ✗ Less stable estimates (higher variance)
- ✗ Throws away useful historical information

Rule of thumb: Start with expanding, switch to rolling if you detect regime changes.

Part 3 : The WalkForwardValidator in Action

The `WalkForwardValidator` automates the entire validation workflow:

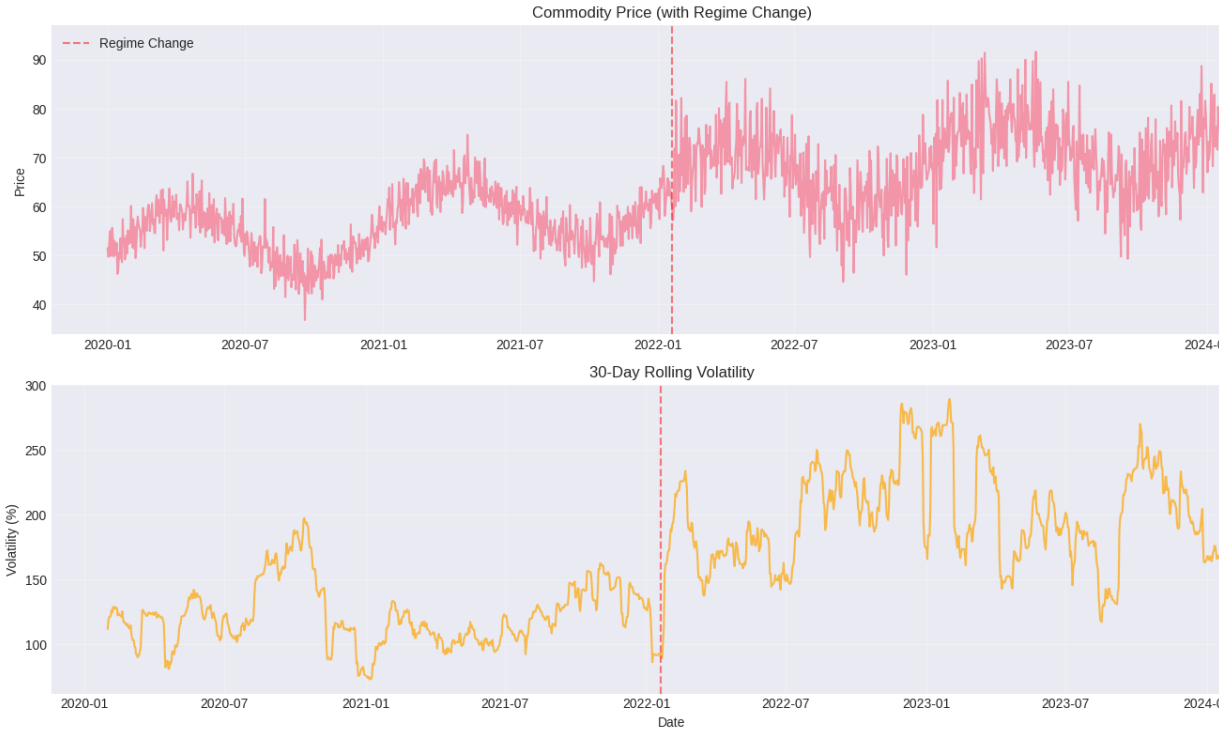
1. Split data into folds
2. Fit model on each training fold
3. Predict on corresponding test fold
4. Calculate metrics
5. Aggregate results across folds

In [7]:

```
# Create realistic commodity dataset
np.random.seed(42)
n = 1500
dates = pd.date_range('2020-01-01', periods=n, freq='D')

# Price with trend, seasonality, and regime changes
trend = np.linspace(50, 75, n)
seasonality = 8 * np.sin(2 * np.pi * np.arange(n) / 365.25)

# Add regime change at midpoint
volatility = np.ones(n) * 3
volatility[n//2:] = 6  # Higher volatility in second half

noise = np.random.normal(0, 1, n) * volatility
price = trend + seasonality + noise

# Create features
inventory = 1000 + np.random.normal(0, 80, n).cumsum()
production = 100 + np.random.normal(0, 4, n)
consumption = 98 + np.random.normal(0, 4, n)

commodity_df = pd.DataFrame({
    'price': price,
    'inventory': inventory,
    'production': production,
    'consumption': consumption,
}, index=dates)

# Add engineered features
commodity_df = create_commodity_features(
    commodity_df,
    price_column='price',
    inventory_column='inventory',
    production_column='production',
    consumption_column='consumption'
)

print(f"✓ Commodity dataset created: {commodity_df.shape}")
print(f"  Features: {len(commodity_df.columns)}")
print(f"  Date range: {commodity_df.index[0].date()} to {commodity_df.ind

# Visualize
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

axes[0].plot(commodity_df.index, commodity_df['price'], alpha=0.7)
axes[0].axvline(dates[n//2], color='red', linestyle='--', alpha=0.5, labe
axes[0].set_ylabel('Price')
axes[0].set_title('Commodity Price (with Regime Change)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Rolling volatility to show regime change
rolling_vol = commodity_df['price'].pct_change().rolling(30).std() * np.s
axes[1].plot(commodity_df.index, rolling_vol, color='orange', alpha=0.7)
axes[1].axvline(dates[n//2], color='red', linestyle='--', alpha=0.5)
axes[1].set_ylabel('Volatility (%)')
axes[1].set_xlabel('Date')
axes[1].set_title('30-Day Rolling Volatility')
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

```
✓ Commodity dataset created: (1500, 40)
  Features: 40
  Date range: 2020-01-01 to 2024-02-08
```



In [8]:
```
# Prepare data for modeling
# Select features (avoid target leakage)
feature_cols = [
    'inventory', 'production', 'consumption',
    'price_lag1', 'price_lag5', 'price_lag21',
    'inventory_lag1', 'inventory_lag5',
    'price_zscore63', 'inventory_zscore52',
    'supply_demand_balance', 'supply_demand_balance_roll4_mean',
    'month', 'quarter'
]

# Filter to existing columns
feature_cols = [c for c in feature_cols if c in commodity_df.columns]

X = commodity_df[feature_cols].dropna()
y = commodity_df.loc[X.index, 'price']

print(f"\nModeling data:")
print(f"  Features: {len(feature_cols)}")
print(f"  Samples: {len(X)}")
print(f"  Feature columns: {feature_cols}")
```

```
Modeling data:
  Features: 11
  Samples: 1495
  Feature columns: ['inventory', 'production', 'consumption', 'inventory_lag1
'inventory_lag5', 'price_zscore63', 'inventory_zscore52', 'supply_demand_bala
'supply_demand_balance_roll4_mean', 'month', 'quarter']
```

In [9]:
```
# Create a simple model wrapper for sklearn compatibility
class SimpleModel:
```

```python
    """Simple model wrapper for validation."""
    def __init__(self, estimator):
        self.estimator = estimator
        self.is_fitted = False

    def fit(self, X, y, as_of_date=None):
        self.estimator.fit(X, y)
        self.is_fitted = True
        return self

    def predict(self, X, as_of_date=None):
        predictions = self.estimator.predict(X)
        return pd.Series(predictions, index=X.index)

    def get_params(self):
        return self.estimator.get_params()

# Create model
model = SimpleModel(Ridge(alpha=1.0))

# Initialize validator
splitter = TimeSeriesSplit(
    n_splits=5,
    train_period='365D',
    test_period='90D',
    gap_period='5D',
    expanding=True
)

validator = WalkForwardValidator(
    splitter=splitter,
    verbose=True,
    store_predictions=True
)

print("✓ Model and validator initialized")
print("\nRunning walk-forward validation...\n")
```

```
✓ Model and validator initialized

Running walk-forward validation...
```

In [10]:

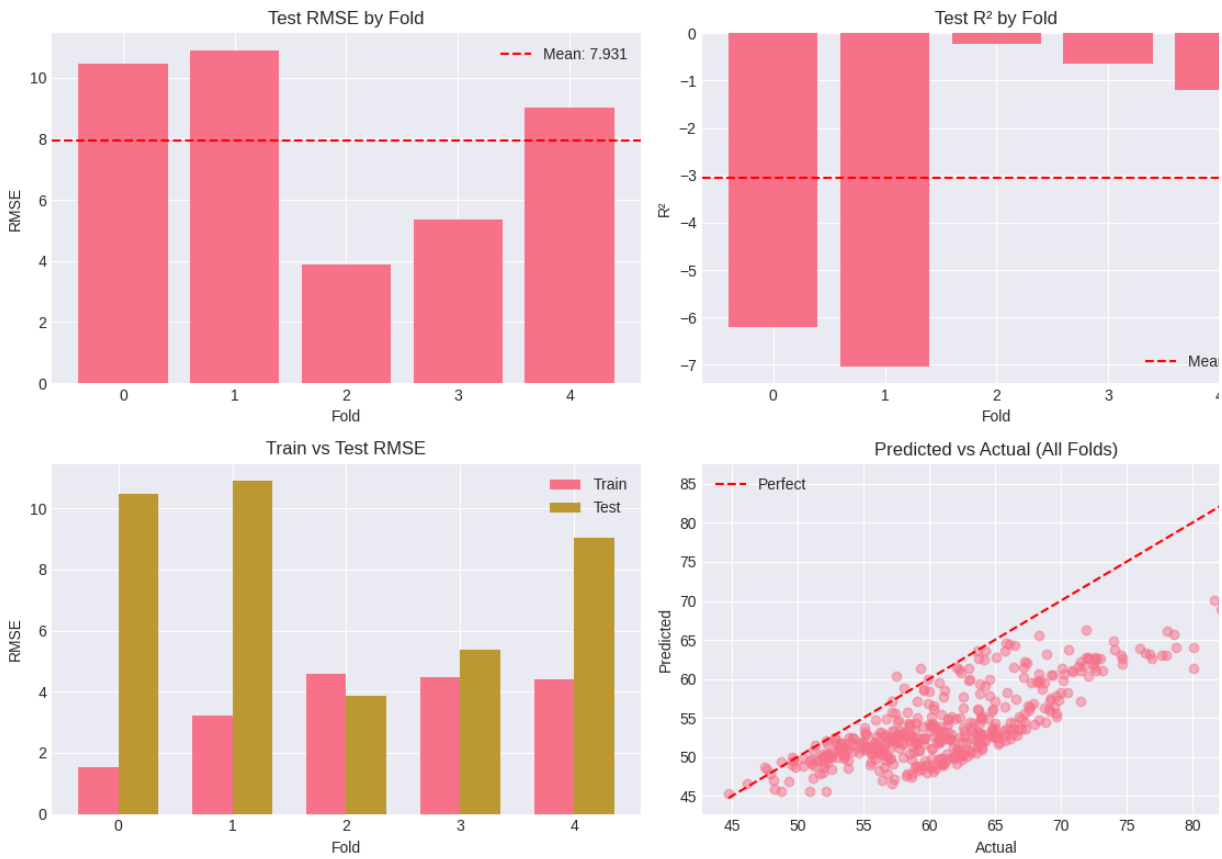
```python
# Run validation
validator.validate(model, X, y)

print("\n✓ Validation complete")
```

```
Processing ValidationFold(fold=0, train=2020-01-06→2021-01-04,
test=2021-01-10→2021-04-09)
Processing ValidationFold(fold=1, train=2020-01-06→2021-04-04,
test=2021-04-10→2021-07-08)
Processing ValidationFold(fold=2, train=2020-01-06→2021-07-03,
test=2021-07-09→2021-10-06)
Processing ValidationFold(fold=3, train=2020-01-06→2021-10-01,
test=2021-10-07→2022-01-04)
Processing ValidationFold(fold=4, train=2020-01-06→2021-12-30,
test=2022-01-05→2022-04-04)

✓ Validation complete
```

In [11]:
```
# Get summary results
summary = validator.get_summary()

print("\nValidation Summary by Fold:")
print(summary[['fold', 'test_rmse', 'test_mae', 'test_r2', 'test_directio

# Get aggregated metrics
agg_metrics = validator.get_aggregated_metrics()

print("\n" + "="*60)
print("AGGREGATED METRICS (ACROSS ALL FOLDS)")
print("="*60)
for metric, value in agg_metrics.items():
    print(f"{metric:30s}: {value:.4f}")
print("="*60)
```

```
Validation Summary by Fold:
   fold  test_rmse  test_mae  test_r2  test_directional_accuracy  test_n_obs
0     0    10.4729    9.9454  -6.2071                     0.9888          90
1     1    10.9074   10.8085  -7.0377                     0.9888          90
2     2     3.8703    3.2559  -0.2280                     0.9888          90
3     3     5.3691    4.6769  -0.6523                     0.8989          90
4     4     9.0340    7.7000  -1.1886                     0.9663          90

============================================================
AGGREGATED METRICS (ACROSS ALL FOLDS)
============================================================
mae_mean                      : 7.2774
mae_std                       : 3.2673
rmse_mean                     : 7.9307
rmse_std                      : 3.1460
r2_mean                       : -3.0627
r2_std                        : 3.2805
directional_accuracy_mean     : 0.9663
directional_accuracy_std      : 0.0389
mape_mean                     : 11.4906
mape_std                      : 4.8819
n_obs_mean                    : 90.0000
n_obs_std                     : 0.0000
============================================================
```

In [12]:
```
# Visualize results
validator.plot_results()
plt.show()
```



Interpreting Fold Results

What to look for:

1. **Train vs Test Performance**:
   - Large gap? Likely overfitting
   - Similar? Good generalization
2. **Consistency Across Folds**:
   - Stable performance? Robust model
   - High variance? Model is unstable or data has regime changes
3. **Directional Accuracy**:
   - > 50%? Model has predictive power
   - ~ 50%? Random, no signal
4. **Temporal Patterns**:
   - Performance degrading over time? Regime change or model decay
   - Performance improving? Market becoming more predictable

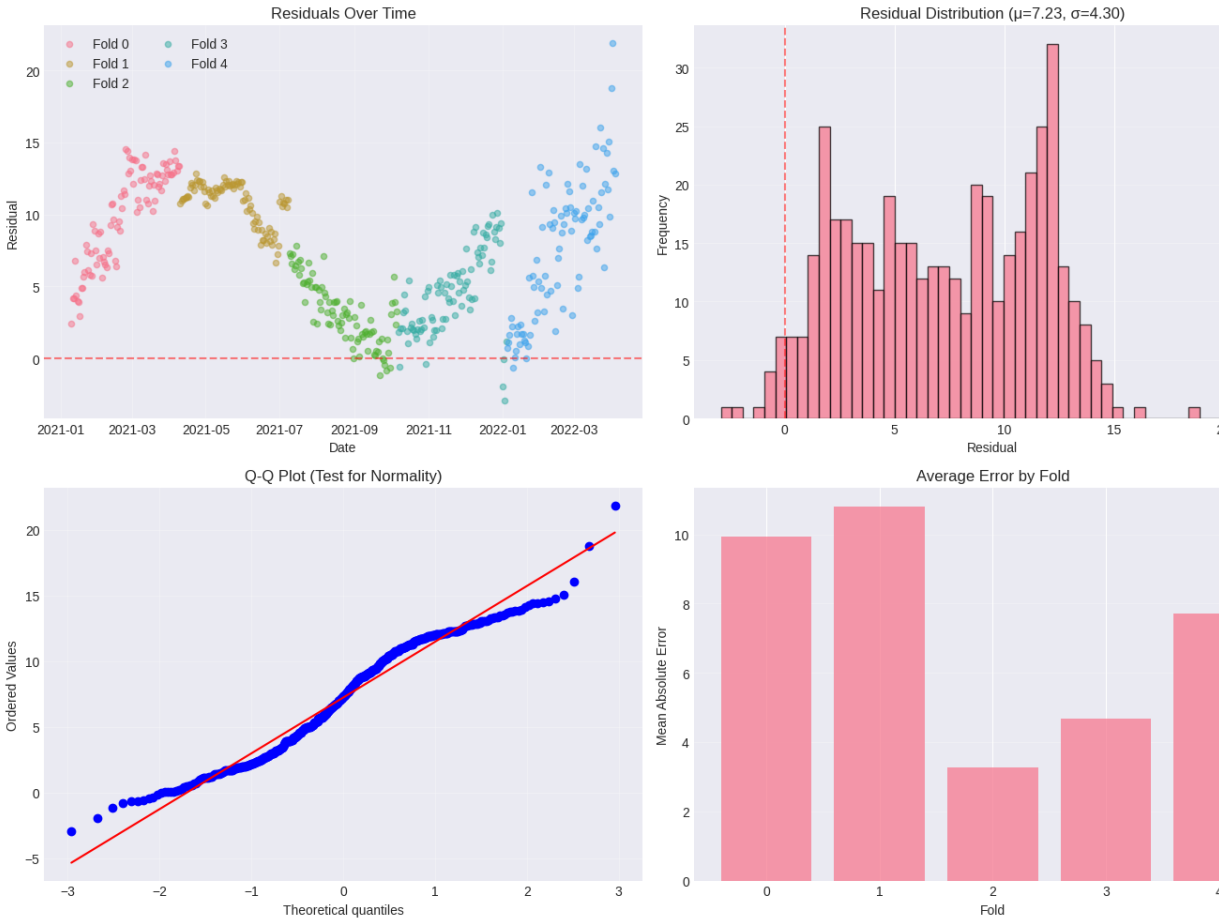
In [13]:
```
# Get all predictions for detailed analysis
all_predictions = validator.get_all_predictions()

print(f"\nPrediction DataFrame shape: {all_predictions.shape}")
print(f"Columns: {list(all_predictions.columns)}")
print("\nSample predictions:")
print(all_predictions.head(10))
```

```
Prediction DataFrame shape: (450, 3)
Columns: ['fair_value', 'fold', 'actual']

Sample predictions:
            fair_value  fold   actual
2021-01-10     52.8579     0  55.2876
2021-01-11     56.1442     0  60.3588
2021-01-12     54.2705     0  58.4436
2021-01-13     57.8098     0  64.6142
2021-01-14     51.4232     0  55.7707
2021-01-15     51.8406     0  55.8266
2021-01-16     52.8018     0  56.6971
2021-01-17     49.3111     0  52.2726
2021-01-18     52.3491     0  57.2153
2021-01-19     51.7343     0  56.6627
```

In [14]:

```
# Analyze residuals by fold
all_predictions['residual'] = all_predictions['actual'] - all_predictions
all_predictions['abs_residual'] = all_predictions['residual'].abs()
all_predictions['pct_error'] = (all_predictions['residual'] / all_predict

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Residuals over time
for fold in all_predictions['fold'].unique():
    fold_data = all_predictions[all_predictions['fold'] == fold]
    axes[0, 0].scatter(fold_data.index, fold_data['residual'], alpha=0.5,
axes[0, 0].axhline(0, color='red', linestyle='--', alpha=0.5)
axes[0, 0].set_ylabel('Residual')
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_title('Residuals Over Time')
axes[0, 0].legend(loc='best', ncol=2)
axes[0, 0].grid(True, alpha=0.3)

# Residual distribution
axes[0, 1].hist(all_predictions['residual'], bins=50, alpha=0.7, edgecolo
axes[0, 1].axvline(0, color='red', linestyle='--', alpha=0.5)
axes[0, 1].set_xlabel('Residual')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].set_title(f"Residual Distribution (μ={all_predictions['residua
axes[0, 1].grid(True, alpha=0.3, axis='y')

# Q-Q plot for normality
from scipy import stats
stats.probplot(all_predictions['residual'].dropna(), dist="norm", plot=ax
axes[1, 0].set_title('Q-Q Plot (Test for Normality)')
axes[1, 0].grid(True, alpha=0.3)

# Absolute error by fold
fold_errors = all_predictions.groupby('fold')['abs_residual'].mean()
axes[1, 1].bar(fold_errors.index, fold_errors.values, alpha=0.7)
axes[1, 1].set_xlabel('Fold')
axes[1, 1].set_ylabel('Mean Absolute Error')
axes[1, 1].set_title('Average Error by Fold')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Statistical tests
```

```python
print("\nResidual Analysis:")
print(f"Mean: {all_predictions['residual'].mean():.4f} (should be ~0)")
print(f"Std: {all_predictions['residual'].std():.4f}")
print(f"Skewness: {all_predictions['residual'].skew():.4f} (should be ~0
print(f"Kurtosis: {all_predictions['residual'].kurtosis():.4f} (should be

# Durbin-Watson test for autocorrelation
from statsmodels.stats.stattools import durbin_watson
dw = durbin_watson(all_predictions.sort_index()['residual'].dropna())
print(f"\nDurbin-Watson statistic: {dw:.4f}")
print(f"  (2 = no autocorrelation, <2 = positive, >2 = negative)")
```




```
Residual Analysis:
Mean: 7.2342 (should be ~0)
Std: 4.3037
Skewness: -0.0004 (should be ~0 for normal)
Kurtosis: -0.8978 (should be ~0 for normal)
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[14], line 52
     49 print(f"Kurtosis: {all_predictions['residual'].kurtosis():.4f} (shoul
~0 for normal)")
     51 # Durbin-Watson test for autocorrelation
---> 52 from statsmodels.stats.stattools import durbin_watson
     53 dw = durbin_watson(all_predictions.sort_index()['residual'].dropna())
     54 print(f"\nDurbin-Watson statistic: {dw:.4f}")

ModuleNotFoundError: No module named 'statsmodels'
```

Part 4 : Detecting Overfitting and Data Leakage

Signs of Overfitting

1. **Large train-test gap** (e.g., train R^2 = 0.95, test R^2 = 0.40)
2. **Degrading performance** across folds
3. **Non-normal residuals** (fat tails, skewness)
4. **High feature importance** for lagged target variables

In [15]:
```
# Calculate overfitting metrics
summary['train_test_gap'] = summary['train_r2'] - summary['test_r2']

print("\nOverfitting Analysis:")
print("\nTrain-Test Performance Gap by Fold:")
print(summary[['fold', 'train_r2', 'test_r2', 'train_test_gap']])

avg_gap = summary['train_test_gap'].mean()
print(f"\nAverage train-test gap: {avg_gap:.4f}")

if avg_gap > 0.15:
    print("⚠️  WARNING: Large train-test gap suggests overfitting")
    print("   Consider: Regularization, feature selection, simpler model"
elif avg_gap > 0.05:
    print("⚠️  CAUTION: Moderate train-test gap")
    print("   Model may not generalize perfectly")
else:
    print("✓ Good generalization (small train-test gap)")

# Visualize
fig, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(summary))
width = 0.35

ax.bar(x - width/2, summary['train_r2'], width, label='Train R²', alpha=0
ax.bar(x + width/2, summary['test_r2'], width, label='Test R²', alpha=0.7
ax.set_xlabel('Fold')
ax.set_ylabel('R²')
ax.set_title('Train vs Test R² by Fold')
ax.set_xticks(x)
ax.set_xticklabels([f'Fold {i}' for i in summary['fold']])
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
Overfitting Analysis:

Train-Test Performance Gap by Fold:
   fold  train_r2  test_r2  train_test_gap
0     0    0.9132  -6.2071          7.1203
1     1    0.7284  -7.0377          7.7662
2     2    0.5088  -0.2280          0.7368
3     3    0.4865  -0.6523          1.1388
4     4    0.4633  -1.1886          1.6519

Average train-test gap: 3.6828
⚠️  WARNING: Large train-test gap suggests overfitting
   Consider: Regularization, feature selection, simpler model
```

In [16]:
```python
# Test for data leakage
leakage_report = detect_data_leakage(
    X=X,
    y=y,
    feature_columns=feature_cols,
    threshold=0.95
)

print("\nData Leakage Detection Report:")
print("="*60)

if leakage_report['is_clean']:
    print("✓ No data leakage detected")
else:
    print("⚠️  POTENTIAL DATA LEAKAGE DETECTED\n")

    if leakage_report['high_correlation_features']:
        print("High Correlation Features (|r| > 0.95):")
        for item in leakage_report['high_correlation_features']:
            print(f"  - {item['column']}: r = {item['correlation']:.4f}")

    if leakage_report['shifted_target_features']:
        print("\nShifted Target Features (look-ahead bias):")
```

```
        for item in leakage_report['shifted_target_features']:
            print(f"  - {item['column']} (shift={item['shift']}): r = {it

print("="*60)
```

```
Data Leakage Detection Report:
============================================================
✓ No data leakage detected
============================================================
```

Part 5 : Model Comparison

Walk-forward validation enables **fair comparison** of multiple models. Let's compare:

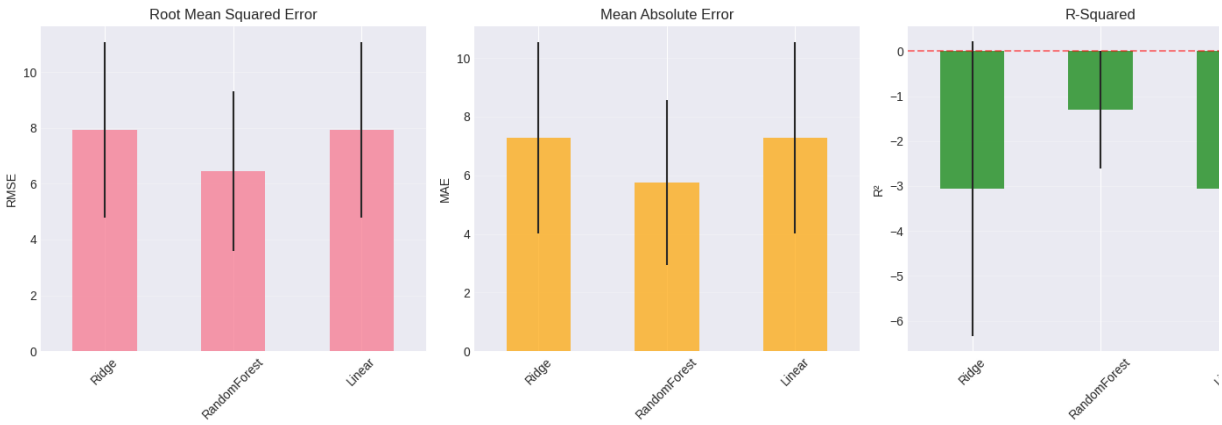
1. Linear model (Ridge)
2. Ensemble model (Random Forest)
3. Naive baseline (last value)

In [17]:
```
# Create multiple models
models = {
    'Ridge': SimpleModel(Ridge(alpha=1.0)),
    'RandomForest': SimpleModel(RandomForestRegressor(n_estimators=50, ma
    'Linear': SimpleModel(LinearRegression()),
}

# Validate each model
results = {}

for name, model in models.items():
    print(f"\nValidating {name}...")

    validator = WalkForwardValidator(
        splitter=splitter,
        verbose=False,
        store_predictions=True
    )

    validator.validate(model, X, y)
    results[name] = validator.get_aggregated_metrics()

    print(f"  RMSE: {results[name]['rmse_mean']:.4f} ± {results[name]['rm
    print(f"  R²: {results[name]['r2_mean']:.4f} ± {results[name]['r2_std

print("\n✓ All models validated")
```

```
Validating Ridge...
  RMSE: 7.9307 ± 3.1460
  R²: -3.0627 ± 3.2805

Validating RandomForest...
  RMSE: 6.4534 ± 2.8492
  R²: -1.3031 ± 1.3067

Validating Linear...
  RMSE: 7.9329 ± 3.1392
  R²: -3.0646 ± 3.2794

✓ All models validated
```

In [18]:
```
# Compare models
comparison_df = pd.DataFrame(results).T

print("\n" + "="*80)
print("MODEL COMPARISON")
print("="*80)
print(comparison_df[['rmse_mean', 'rmse_std', 'mae_mean', 'r2_mean', 'dir
print("="*80)

# Find best model
best_model = comparison_df['rmse_mean'].idxmin()
print(f"\n🏆 Best Model: {best_model} (lowest RMSE)")

# Visualize comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# RMSE
comparison_df['rmse_mean'].plot(kind='bar', ax=axes[0], alpha=0.7, yerr=c
axes[0].set_ylabel('RMSE')
axes[0].set_title('Root Mean Squared Error')
axes[0].grid(True, alpha=0.3, axis='y')
axes[0].tick_params(axis='x', rotation=45)

# MAE
comparison_df['mae_mean'].plot(kind='bar', ax=axes[1], alpha=0.7, yerr=co
axes[1].set_ylabel('MAE')
axes[1].set_title('Mean Absolute Error')
axes[1].grid(True, alpha=0.3, axis='y')
axes[1].tick_params(axis='x', rotation=45)

# R²
comparison_df['r2_mean'].plot(kind='bar', ax=axes[2], alpha=0.7, yerr=com
axes[2].set_ylabel('R²')
axes[2].set_title('R-Squared')
axes[2].axhline(0, color='red', linestyle='--', alpha=0.5)
axes[2].grid(True, alpha=0.3, axis='y')
axes[2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

```
================================================================================
MODEL COMPARISON
================================================================================
              rmse_mean  rmse_std  mae_mean  r2_mean  \
Ridge            7.9307    3.1460    7.2774  -3.0627
RandomForest     6.4534    2.8492    5.7489  -1.3031
Linear           7.9329    3.1392    7.2794  -3.0646

              directional_accuracy_mean
Ridge                            0.9663
RandomForest                     0.8989
Linear                           0.9663
================================================================================

🏆 Best Model: RandomForest (lowest RMSE)
```



Statistical Significance Testing

Is one model **statistically significantly** better than another?

In [19]:
```
# Collect fold-level metrics for statistical testing
fold_metrics = {}

for name, model in models.items():
    validator = WalkForwardValidator(splitter=splitter, verbose=False)
    validator.validate(model, X, y)
    summary = validator.get_summary()
    fold_metrics[name] = summary['test_rmse'].values

# Paired t-test (since same folds)
from scipy.stats import ttest_rel

print("\nPaired T-Test Results (comparing RMSE):")
print("="*60)

model_names = list(fold_metrics.keys())
for i in range(len(model_names)):
    for j in range(i+1, len(model_names)):
        name1, name2 = model_names[i], model_names[j]
        metric1, metric2 = fold_metrics[name1], fold_metrics[name2]

        t_stat, p_value = ttest_rel(metric1, metric2)

        print(f"\n{name1} vs {name2}:")
        print(f"  t-statistic: {t_stat:.4f}")
        print(f"  p-value: {p_value:.4f}")

        if p_value < 0.05:
            better = name1 if t_stat < 0 else name2
            print(f"  ✓ Statistically significant difference (α=0.05)")
            print(f"    {better} is significantly better")
        else:
            print(f"  ✗ No significant difference (α=0.05)")

print("="*60)
```

```
Paired T-Test Results (comparing RMSE):
============================================================

Ridge vs RandomForest:
  t-statistic: 0.9577
  p-value: 0.3925
  ✗ No significant difference (α=0.05)

Ridge vs Linear:
  t-statistic: -0.3926
  p-value: 0.7146
  ✗ No significant difference (α=0.05)

RandomForest vs Linear:
  t-statistic: -0.9589
  p-value: 0.3919
  ✗ No significant difference (α=0.05)
============================================================
```

Part 6 : Model Decay Analysis

Models **degrade over time** as:

- Market regimes change
- Relationships break down
- New information becomes available

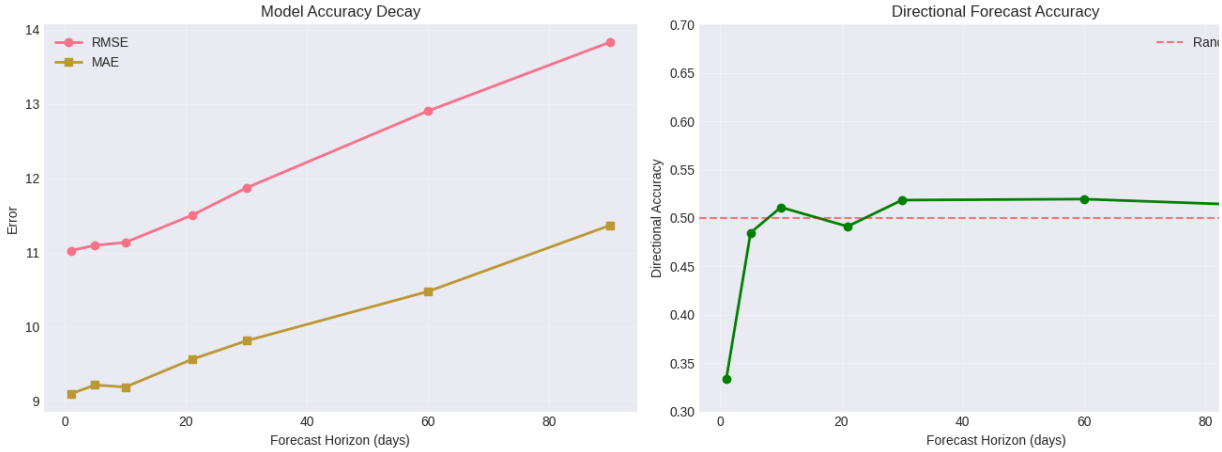
Understanding decay helps determine **retraining frequency**.

In [20]:

```
# Analyze model decay
# Fit model once, test at increasing horizons

train_end_date = X.index[int(len(X) * 0.6)]  # Use 60% for training
X_train = X.loc[:train_end_date]
y_train = y.loc[:train_end_date]
X_test = X.loc[train_end_date:]
y_test = y.loc[train_end_date:]

# Fit once
decay_model = SimpleModel(Ridge(alpha=1.0))
decay_model.fit(X_train, y_train)
predictions = decay_model.predict(X_test)

# Calculate decay at different horizons
decay_results = calculate_model_decay(
    predictions=pd.DataFrame({'fair_value': predictions}),
    actuals=y_test,
    horizons=[1, 5, 10, 21, 30, 60, 90]
)

print("\nModel Decay Analysis:")
print(decay_results)

# Visualize decay
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Error metrics
axes[0].plot(decay_results['horizon'], decay_results['rmse'], 'o-', label
```

```
axes[0].plot(decay_results['horizon'], decay_results['mae'], 's-', label=
axes[0].set_xlabel('Forecast Horizon (days)')
axes[0].set_ylabel('Error')
axes[0].set_title('Model Accuracy Decay')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Directional accuracy
axes[1].plot(decay_results['horizon'], decay_results['directional_accurac
axes[1].axhline(0.5, color='red', linestyle='--', alpha=0.5, label='Rando
axes[1].set_xlabel('Forecast Horizon (days)')
axes[1].set_ylabel('Directional Accuracy')
axes[1].set_title('Directional Forecast Accuracy')
axes[1].set_ylim([0.3, 0.7])
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Find decay half-life (where performance drops 50%)
rmse_initial = decay_results.iloc[0]['rmse']
rmse_threshold = rmse_initial * 1.5
decay_horizon = decay_results[decay_results['rmse'] > rmse_threshold]

if len(decay_horizon) > 0:
    print(f"\n⚠️  Model performance degrades significantly after {decay_h
    print(f"   Consider retraining every {decay_horizon.iloc[0]['horizon'
else:
    print("\n✓ Model remains stable over tested horizons")
```

```
Model Decay Analysis:
   horizon      mae     rmse  directional_accuracy  n_observations
0        1   9.0955  11.0245                0.3339             597
1        5   9.2150  11.0955                0.4848             593
2       10   9.1871  11.1341                0.5111             588
3       21   9.5604  11.5006                0.4913             577
4       30   9.8099  11.8696                0.5185             568
5       60  10.4749  12.9054                0.5196             538
6       90  11.3640  13.8306                0.5128             508
```




```
✓ Model remains stable over tested horizons
```

Part 7: Best Practices and Common Pitfalls

✓ Best Practices

1. **Always use temporal splits** - Never random shuffle for time series
2. **Include gap periods** - Prevent leakage from data revisions
3. **Match validation to trading** - Test period = your holding period
4. **Check train-test gap** - Large gap = overfitting
5. **Test multiple models** - Use statistical tests for comparison
6. **Analyze residuals** - Should be white noise
7. **Monitor model decay** - Determines retraining frequency

✗ Common Pitfalls

1. **Using sklearn's KFold** - Causes severe look-ahead bias
2. **No gap period** - Allows information leakage
3. **Single fold validation** - Doesn't test robustness
4. **Ignoring regime changes** - Model breaks in new regime
5. **Over-tuning on validation set** - Validation becomes training
6. **Forgetting about transaction costs** - Real profit < backtest
7. **Not testing at multiple horizons** - Miss decay patterns

The Production Checklist

Before deploying a model:

- ☐ Walk-forward validation complete with realistic splits?
- ☐ Gap period included to prevent leakage?
- ☐ Train-test performance gap acceptable (< 10%)?
- ☐ Residuals approximately normal and uncorrelated?
- ☐ Model stable across different time periods?
- ☐ Statistical significance vs baseline confirmed?
- ☐ Model decay analyzed and retraining schedule set?
- ☐ Transaction costs and slippage accounted for?
- ☐ Data availability confirmed in production?
- ☐ Monitoring system in place for live performance?

Exercises

1. **Implement adaptive validation**: Modify the validator to use shorter refit periods during high

2. **Create a model ensemble**: Use walk-forward validation to optimize ensemble weights
3. **Regime detection**: Identify market regimes and validate models separately in each regime
4. **Bootstrap validation**: Implement block bootstrap for more robust uncertainty estimates
5. **Rolling vs Expanding**: Compare performance of rolling and expanding windows for your co

Key Takeaways

1. **Random cross-validation is catastrophically wrong for time series** - It creates impossib scenarios and inflates performance
2. **Walk-forward validation mimics production** - Train on past, predict on future, with proper temporal order
3. **Gap periods prevent leakage** - Account for data revisions and information delays
4. **Expanding vs Rolling windows** - Trade off between more data and adaptability
5. **The `WalkForwardValidator` automates proper validation** - Handles splits, prediction metrics
6. **Train-test gaps reveal overfitting** - Large gaps mean model won't generalize
7. **Statistical tests enable fair model comparison** - Don't rely on point estimates
8. **Model decay analysis determines retraining frequency** - Performance degrades over tim
9. **Validation is about finding reality, not confirming hopes** - Be skeptical of good results

Next Steps

In **Module 8: Signal Discovery and Alpha Generation**, we'll learn how to convert fair value estimates into actionable trading signals with proper risk management.